

BÀI BÁO CÁO TỔNG HỢP

CNC 6 TRỰC

Biên soạn: Cao Tuấn Minh

Mục Lục

1. Bài báo cáo dự án 1.....	1
1.1. Giới thiệu dự án 1	1
1.2. Tổng quan chương trình.....	1
1.2.1. Giới thiệu giao diện và tên gọi	1
1.2.2. Giới hạn chương trình và các thông số mặc định.....	2
1.3. Giai đoạn 1: Input lệnh G-Code	4
1.3.1. Flowchart giai đoạn 1	4
1.3.2. Input lệnh G-Code	5
1.4. Giai đoạn 2: Tiền xử lý lệnh G-Code đầu vào.....	6
1.4.1. Flowchart giai đoạn 2	6
1.4.2. Tiền xử lý lệnh G-Code đầu vào.....	7
1.5. Giai đoạn 3: Xử lý lệnh G-Code đầu vào	8
1.5.1. Flowchart giai đoạn 3	8
1.5.2. Xử lý lệnh G90, G91	8
1.5.3. Bài toán 1 micro giây	9
1.6. Giai đoạn 3.1: Xử lý lệnh G00, G01	14
1.6.1. Flowchart giai đoạn 3.1	14
1.6.2. Xử lý lệnh G00, G01	16
1.6.3. Mô phỏng lệnh G00, G01	21
1.7. Giai đoạn 3.2: Xử lý lệnh G02, G03	32
1.7.1. Flowchart giai đoạn 3.2	32
1.7.2. Xử lý lệnh G02, G03.....	35
1.7.3. Mô phỏng lệnh G02, G03	47
3. Bài báo cáo dự án 3.....	51
3.1. Giới thiệu dự án 3	51
3.2. Nguyên lý hoạt động của động cơ bước	51
3.2.1. Khái niệm	51
3.2.2. Cấu tạo	51
3.2.3. Nguyên lý hoạt động.....	52
3.3. Tìm hiểu các chế độ của động cơ bước	52
3.3.1. Chế độ Full Step	52
3.3.2. Chế độ Half Step.....	53

3.3.3.	Microstep (Vi Bước).....	54
3.4.	Tìm hiểu các tín hiệu điều khiển cơ bản của driver	55
3.4.1.	STEP	55
3.4.2.	DIRECTION	56
3.4.3.	ENABLE	57
3.4.4.	Pulse Width	57
3.4.5.	Logic Level.....	58
3.4.6.	Timing Requirement	58
3.5.	Phương pháp tạo tín hiệu điều khiển	59
3.5.1.	Phương pháp Bresenham	59
3.6.	Thiết kế phần cứng	61
3.6.1.	Danh sách thiết bị	61
3.6.2.	Sơ đồ mạch (Schematic)	61
3.7.	Thiết kế phần mềm	62
3.8.	Điều khiển động cơ.....	66
3.8.1.	Timing Lượng hóa STEP trên tọa độ	66
3.8.2.	Tìm giới hạn Resolution Time	66
3.8.3.	Thông tin và giới hạn chương trình hiện tại:.....	68

3. Bài báo cáo dự án 3

3.1. Giới thiệu dự án 3

- **Thời gian giao nhiệm vụ:** 24/07/2026
- **Thời gian báo cáo:**
- **Thành viên thực hiện:** Cao Tuấn Minh, Hoàng Trọng Trí.
- **Nhiệm vụ:** Tìm hiểu tín hiệu điều khiển của Driver động cơ bước và các phương thức để tạo ra các tín hiệu đó. Thực hiện báo cáo với các trường hợp để điều khiển khi sử dụng MCU để tạo xung điều khiển nhiều motor cùng một lúc.

3.2. Nguyên lý hoạt động của động cơ bước

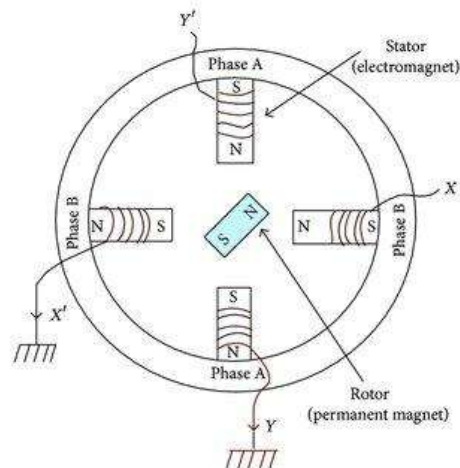
3.2.1. Khái niệm

- **Động cơ bước (Stepper Motor):** là loại động cơ điện *quay theo từng góc cố định (step)* thay vì quay liên tục như động cơ DC thông thường.
- **Ví dụ:**
 - Động cơ có góc bước là 1.8° .
 - Từ đó, số bước trong một vòng quay là:
$$\frac{360^\circ}{1.8^\circ} = 200 \text{ (bước)}$$
 - Vì thế, với 1 xung điều khiển, ta quay được 1 bước.
 - Với 200 xung điều khiển, ta quay được đúng 1 vòng.
 - Nhờ việc đếm số xung, ta có thể biết được động cơ đó đã quay bao nhiêu vòng.

3.2.2. Cấu tạo

- Một động cơ bước bao gồm:
 - **Rotor:** là thành phần chuyển động, được làm từ nam châm vĩnh cửu hoặc vật liệu từ.
 - **Stator:** là thành phần đứng yên, nhiều cuộn dây được chia thành các pha (A, B, ...).

3.2.3. Nguyên lý hoạt động



- **Ví dụ:** ta có một động cơ bước 2 pha (A, B) như trên, khi ta cấp điện lần lượt vào các cuộn dây:

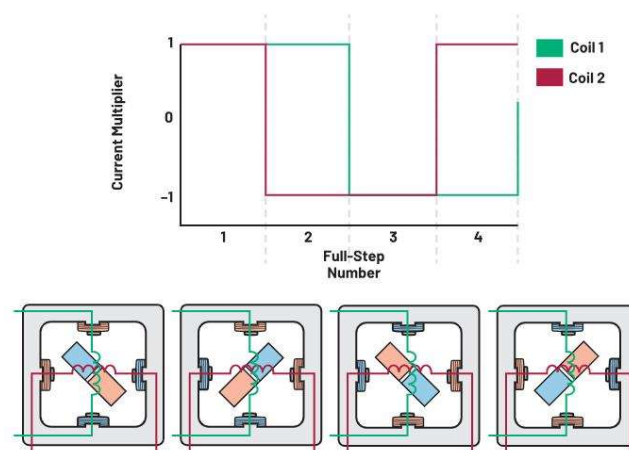
$$A \rightarrow B \rightarrow A' \rightarrow B'$$

- **Khi điện được cấp vào một cuộn dây:** sinh ra từ trường xoay. Thành phần Rotor luôn bị hút về từ trường này, hay nói cách khác, Rotor sẽ tiếp tục chạy theo từ trường.
- Cứ thế, mỗi lần đổi trạng thái, rotor quay một góc cố định, và khi việc này lặp lại nhiều lần, động cơ quay.

3.3. Tìm hiểu các chế độ của động cơ bước

3.3.1. Chế độ Full Step

- **Khái niệm:** đây là chế độ đơn giản nhất, với mỗi xung Step, động cơ quay được đúng 1 bước đầy đủ.



▪ **Ví dụ:**

- Động cơ có *full step* là 1.8° .
- Khi kích 1 xung:

Motor quay được 1.8°

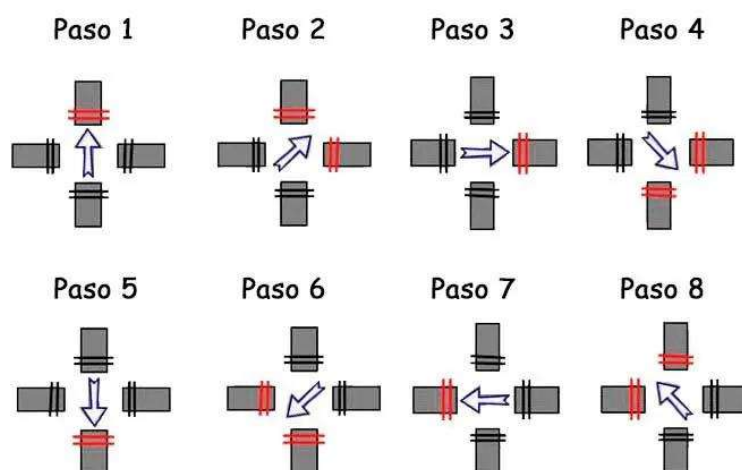
- **Đặc điểm:** với chế độ này, góc bước lớn sẽ dễ điều khiển, đồng thời, sinh ra momen lớn.
- **Ưu điểm:** ở chế độ này điều khiển sẽ đơn giản hơn, momen sẽ đạt giá trị lớn và ít yêu cầu driver.
- **Nhược điểm:** độ phân giải thấp, chạy không mịn, hơi giật và rung nhiều.

3.3.2. Chế độ Half Step

- **Khái niệm:** đây là chế độ xen kẽ giữa 1 pha và 2 pha. Nhờ thế, mà Rotor có thể quay nhiều bước hơn trong một vòng quay.

Secuencia Paso a Paso con dos bobinas, Medio Paso

Paso	A	B	C	D
Paso 1	1	0	0	0
Paso 2	1	1	0	0
Paso 3	0	1	0	0
Paso 4	0	1	1	0
Paso 5	0	0	1	0
Paso 6	0	0	1	1
Paso 7	0	0	0	1
Paso 8	1	0	0	1



▪ **Ví dụ:**

- Động cơ có *full step* là 1.8° . Từ đó, suy ra *half step* là 0.9° .
- Khi kích 1 xung:

Motor quay được 0.9°

- **Ưu điểm:** ở chế độ này, độ phân giải của motor được tăng gấp đôi, chuyển động mượt hơn so với chế độ Full Step. Từ đó, độ rung giảm và đạt độ chính xác vị trí tốt hơn.
- **Nhược điểm:** điều khiển phức tạp hơn, momen xoắn không đều giữa các bước (bước chỉ kích 1 pha có momen thấp hơn bước kích 2 pha nếu không bù dòng). Tần số xung cần cao gấp đôi để đạt cùng tốc độ quay như chế độ Full Step.

3.3.3. Microstep (Vi Bước)

- Với mục tiêu giúp cho Rotor có thể quay mượt hơn, ta cần làm cho từ trường thay đổi liên tục.
- **Ví dụ:**
 - Động cơ có *full step* là 1.8° .
 - Nếu driver đặt giá trị vi bước là $\frac{1}{16}$.
 - Động cơ đó sẽ có vi góc trên một bước góc là:

$$\frac{1.8^\circ}{16} = 0.1125^\circ$$

- Với chế độ Half Step như được đề cập ở trên, có được nhờ việc nhân đôi từ 1 STEP thành 2 STEP, để có thể cho Rotor quay mịn và chính xác hơn, ta cần nhân đôi số STEP lên.
- **Tại sao lại nhân đôi:** nhờ việc nhân đôi, ta chỉ cần tăng thêm **1 bit** để tăng gấp đôi độ phân giải khi:

$$\text{Số STEP} = 2^n, \text{ với } n \text{ là số Bit}$$

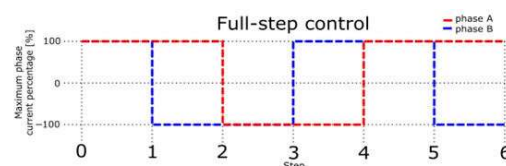
Số Bit	Số STEP
1	2
2	4
3	8
4	16
5	32

- **Ưu điểm:** nhờ có đại lượng này, Rotor chuyển động rất mượt, giảm rung, tiếng ồn, và hiện tượng cộng hưởng. Độ phân giải điều khiển cao phù hợp cho máy CNC, máy in 3D, ...
- **Nhược điểm:** Driver và thuật toán điều khiển sẽ phức tạp hơn. Momen xoắn trên từng microstep nhỏ hơn Full Step, vì thế cũng ảnh hưởng ít nhiều đến độ chính xác vị trí đặc biệt là khi có tải hoặc ma sát. MCU phải phát xung Step với tần số cao hơn để đạt cùng tốc độ quay so với Full Step.

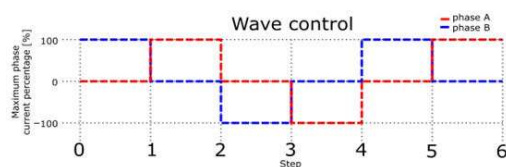
3.4. Tìm hiểu các tín hiệu điều khiển cơ bản của driver

3.4.1. STEP

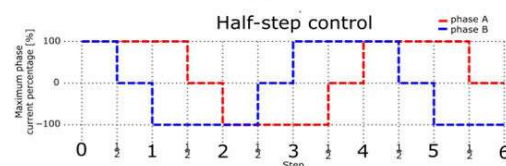
- **Khái niệm:** đây là một chân nhận xung điều khiển của driver từ vi điều khiển. Mỗi khi driver phát hiện một cạnh lên (*Rising Edge*) của tín hiệu STEP (0 đến 1), động cơ sẽ di chuyển một bước điều khiển.
- **Ví dụ:**
 - Full Step → 1 xung = 1 bước (1.8°).
 - Half Step → 1 xung = 0.9° .
 - Vi bước 1/16 → 1 xung = 1/16 bước.



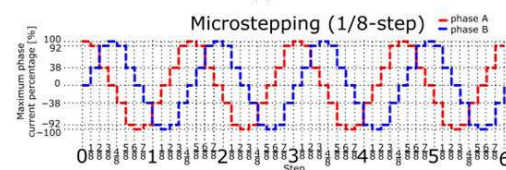
(a)



(b)



(c)



(d)

- **Đặc điểm:**

- **Tốc độ quay phụ thuộc vào STEP:** Khi động cơ quay 200 (bước/vòng). Giả sử, động cơ được cấu hình vi bước là 32, vì thế, để quay được một vòng, ta cần cấp:

$$32 \times 200 = 6400 \text{ xung/vòng}$$

, nếu MCU phát **200 (Hz)**, hay:

$$200 \text{ xung/giây} = 200 \text{ (Hz)}$$

, với tần số phát của MCU, ta nhận thấy 1 giây quay được $\frac{1}{32}$ vòng và có tần số quay của động cơ tương ứng là:

$$\frac{1}{32} \text{ vòng/giây} = 1.875 \text{ RPM}$$

, mặt khác, nếu MCU phát **32 (KHz)**, hay:

$$32000 \text{ xung/giây}$$

, với tần số phát của MCU, ta nhận thấy 1 giây quay được 5 vòng và có tần số quay của động cơ tương ứng là:

$$5 \text{ vòng/giây} = 300 \text{ RPM}$$

- **Ưu điểm:** điều khiển đơn giản, chỉ cần timer của MCU, ta có thể điều khiển chính xác vị trí bằng cách đếm số xung.
- **Nhược điểm:** nếu MCU tạo xung sai, động cơ sẽ chạy sai. Và nếu xung quá nhanh, động cơ có thể bị mất bước (*miss step*).

3.4.2. DIRECTION

- **Khái niệm:** đây là tín hiệu quyết định chiều quay. Khi tín hiệu DIRECTION được đặt, một khoảng thời gian ngắn tiếp theo driver mới phát STEP.
- **Thông thường,**
 - **DIRECTION = LOW:** quay theo chiều kim đồng hồ (CW).
 - **DIRECTION = HIGH:** quay theo ngược chiều kim đồng hồ (CCW).
- **Ưu điểm:** điều khiển đơn giản thông qua 1 GPIO.
- **Nhược điểm:** việc đổi DIRECTION khi đang phát STEP có thể gây sai bước nếu không tuân thủ thời gian thiết lập (*setup time*).

3.4.3. ENABLE

- **Khái niệm:** đây là tín hiệu dùng để bật hoặc tắt driver. Khi thiết lập ở HIGH, thông thường, driver sẽ ngừng cấp dòng cho cuộn dây. Vì thế, động cơ không còn momen giữ và có thể xoay trục Rotor bằng tay.
- **Thông thường,**
 - **ENABLE = LOW:** driver hoạt động.
 - **ENABLE = HIGH:** driver ngừng cấp dòng.
- **Ứng dụng:** giúp tiết kiệm điện, giảm nhiệt, dừng khẩn cấp hoặc khởi tạo hệ thống.
- **Ưu điểm:** tiết kiệm công suất, giảm nhiệt độ động cơ và driver, từ đó có thể bật tắt nhiều động cơ độc lập.
- **Nhược điểm:** Khi Disable, ngừng cấp dòng, ta có thể làm trục dịch chuyển vì mất momen giữ.

3.4.4. Pulse Width

- **Khái niệm:** hay độ rộng xung, là khoảng thời gian tín hiệu STEP ở mức HIGH hoặc LOW.
- **Pulse Width tối thiểu:** một driver cần đủ thời gian để:
 - Phát hiện cạnh xung.
 - Xử lý tín hiệu.
 - Cập nhật bộ tạo vi bước.
 - Điều khiển dòng qua cuộn dây.
 - Nếu xung quá ngắn, khi MCU tạo xung xong, driver chưa kịp nhận. Từ đó, động cơ bị mất bước.
- **Datasheet:** vì thế, cần phải xem trong datasheet ở cái giá trị
 - **Minimum HIGH:** 1.9 (μs), nên thời gian cho mức HIGH phải lớn hơn hoặc bằng 1.9 (μs).
 - **Minimum LOW:** 1.9 (μs), nên thời gian cho mức LOW phải lớn hơn hoặc bằng 1.9 (μs).
- **Ưu điểm:** khi Pulse Width phù hợp, driver nhận xung ổn định, giảm lỗi, tránh trường hợp mất bước.

- **Nhược điểm:** khi Pulse Width quá lớn, driver nhận xung không ổn định. Từ đó, tần số STEP cực đại bị giới hạn và tốc độ tối đa của động cơ cũng từ đó mà bị giới hạn.

3.4.5. Logic Level

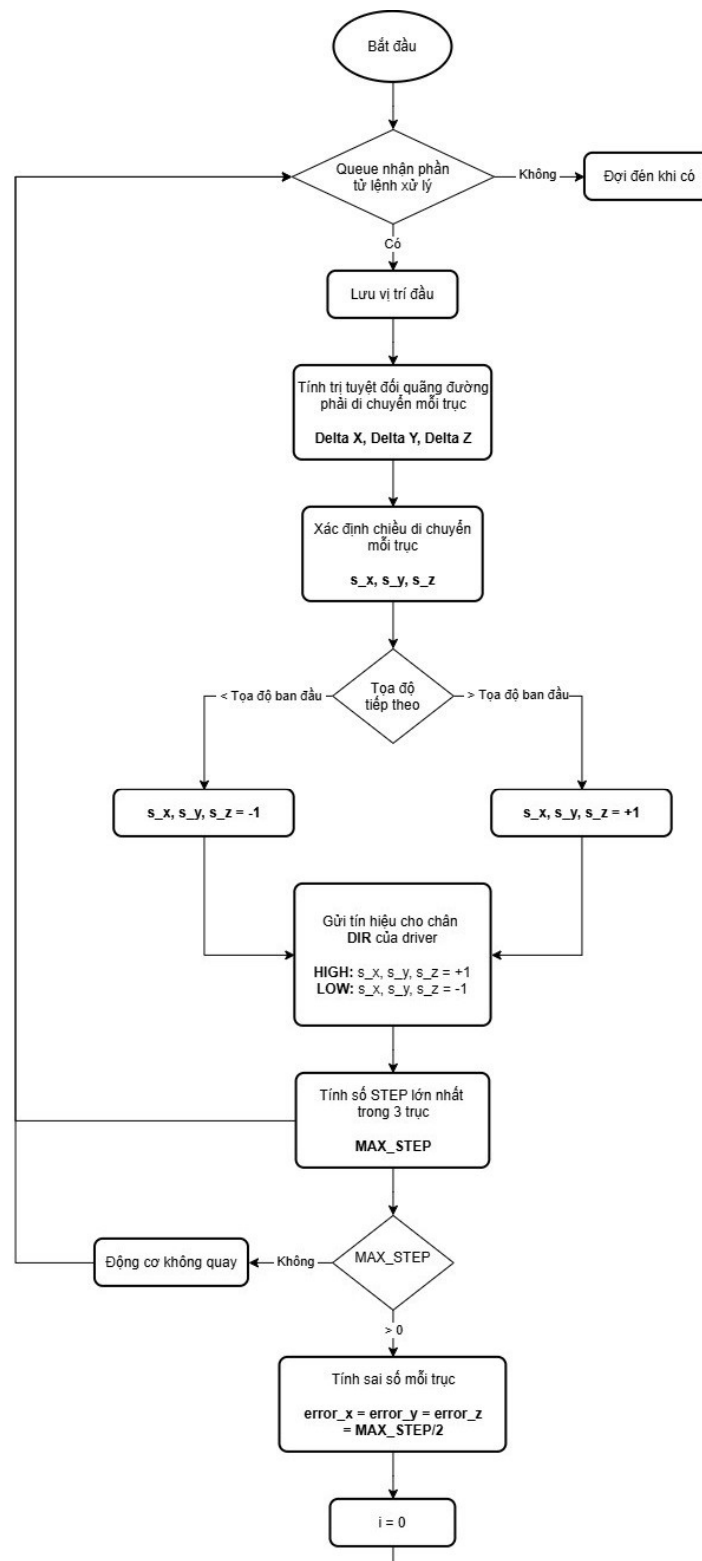
- **Khái niệm:** là mức điện áp mà driver hiểu là:
 - **Logic LOW:** là mức điện áp 0V.
 - **Logic HIGH:** thông thường có thể là mức điện áp 3.3V hoặc 5V, tùy từng thiết bị. Có những driver yêu cầu Logic HIGH khi mức điện áp lớn hơn hoặc bằng một mức điện áp cụ thể.
- **Datasheet:** ta cần kiểm tra datasheet về mức Logic HIGH của driver xem có phù hợp với mức Logic HIGH của MCU. Nếu có sự chênh lệch hoặc không thỏa, ta cần sử dụng bộ chuyển mức Logic (*Logic Level Shifter*).

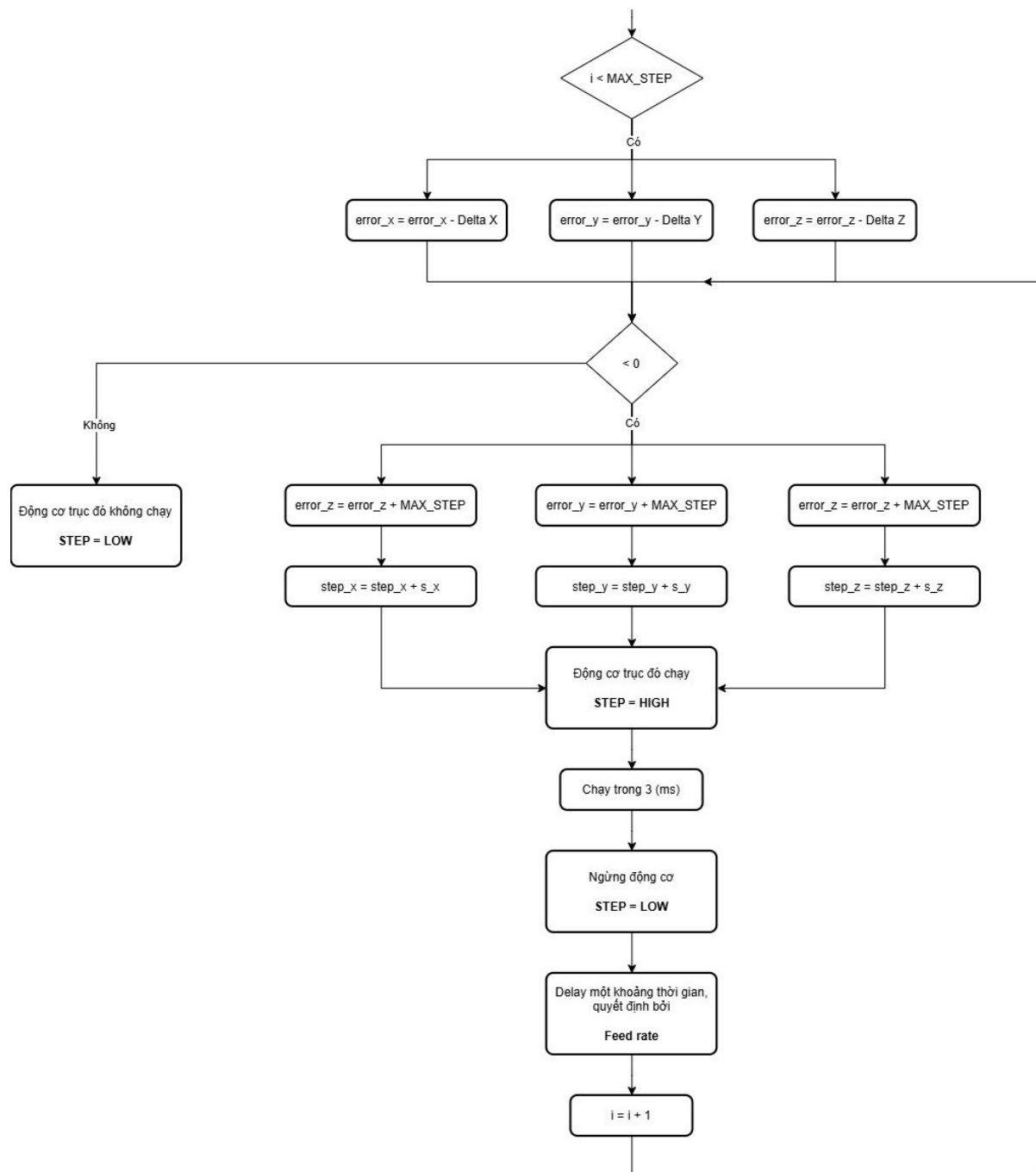
3.4.6. Timing Requirement

- **Khái niệm:** là các ràng buộc thời gian mà MCU phải tuân thủ theo khi điều khiển driver. Nó bao gồm:
 - **Setup Time**
 - **Hold Time**
 - **Pulse Width**
 - **Delay Enable**
 - **Delay khi đổi DIRECTION**
- **Datasheet:** từ các ràng buộc trên, ta phải luôn kiểm tra từ datasheet của driver để có thể thiết lập driver phù hợp.
- **Ưu điểm:** nhờ việc cấu hình phù hợp, driver hoạt động ổn định, đảm bảo không mất bước do vi phạm thời gian.
- **Nhược điểm:** đòi hỏi ngược lại với MCU hoặc bộ tạo xung phải có khả năng tạo xung với độ chính xác cao đặc biệt là khi điều khiển nhiều động cơ tốc độ lớn.

3.5. Phương pháp tạo tín hiệu điều khiển

3.5.1. Phương pháp Bresenham



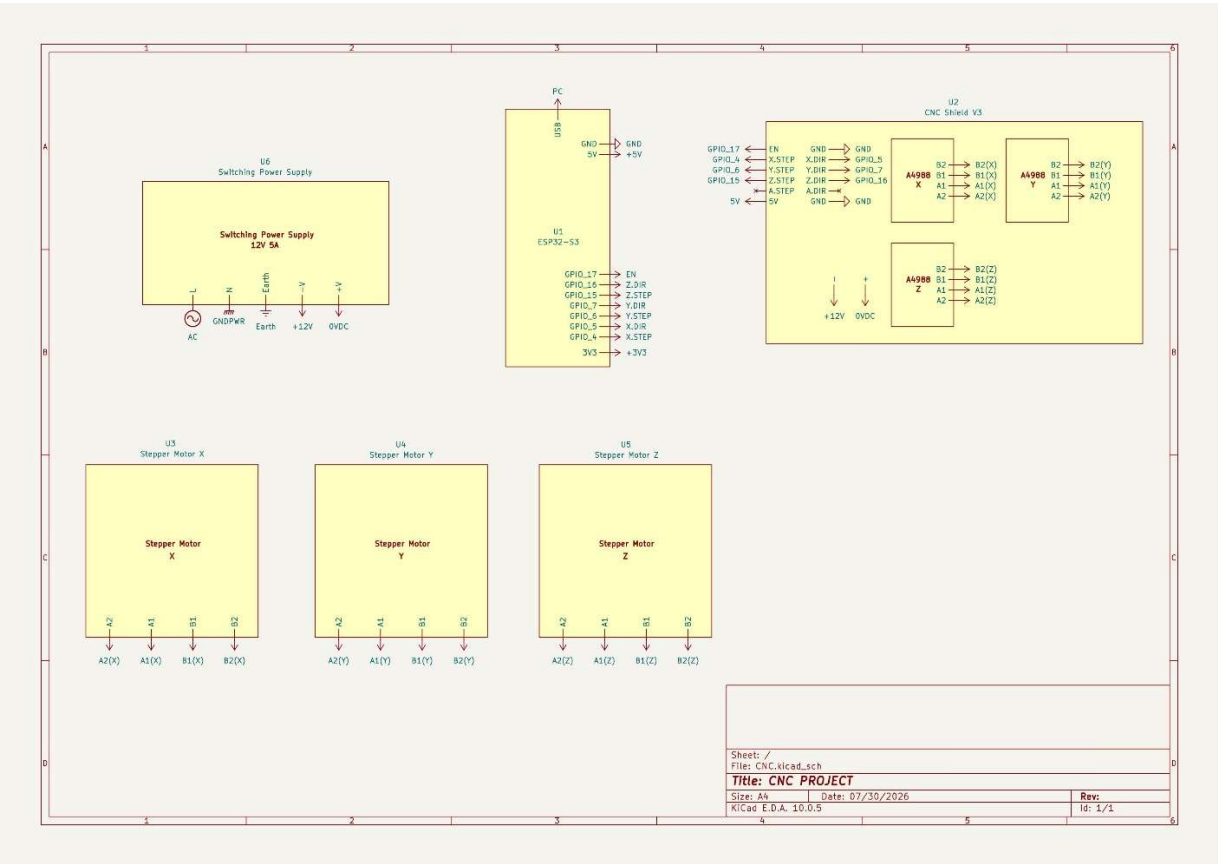


3.6. Thiết kế phần cứng

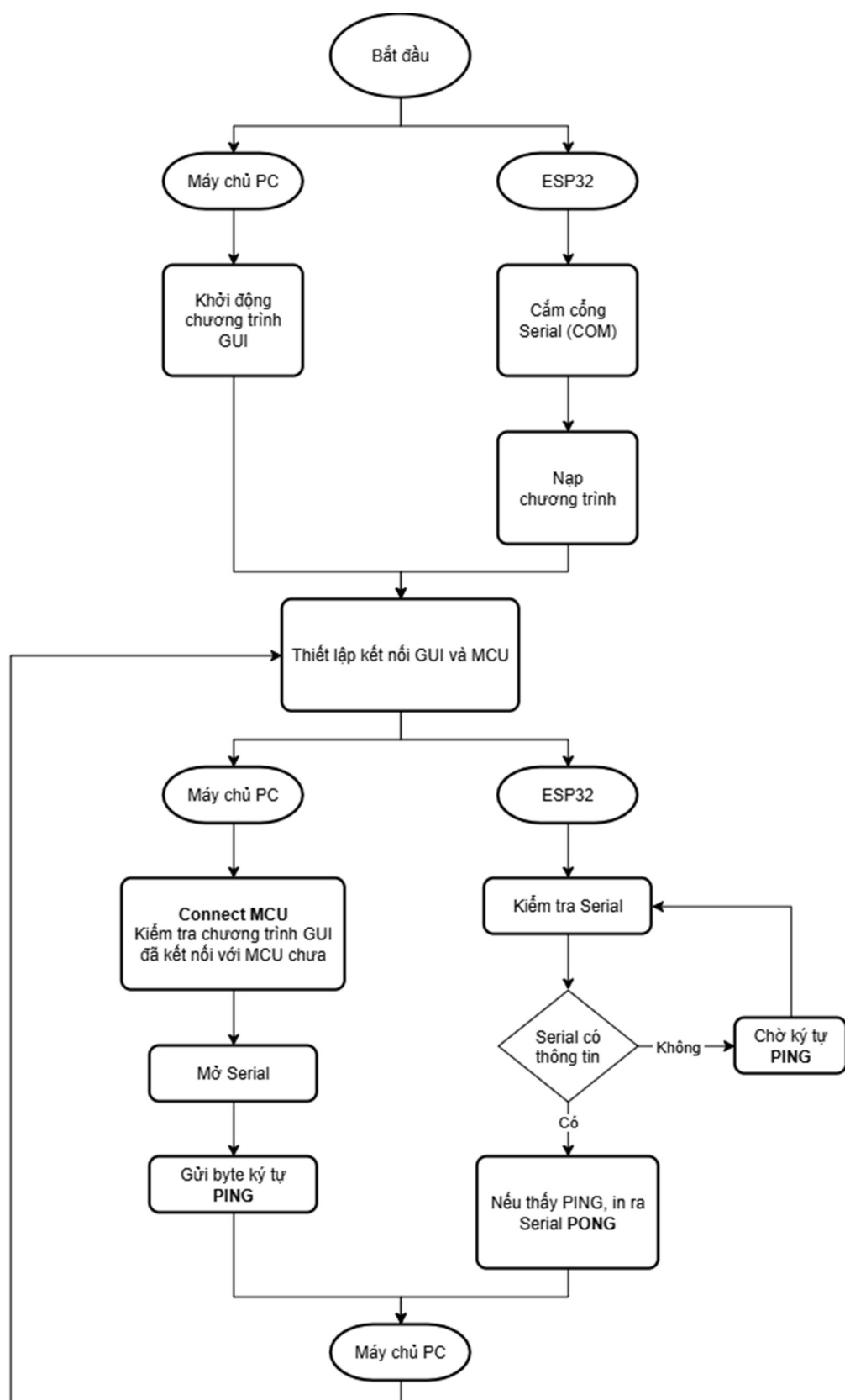
3.6.1. Danh sách thiết bị

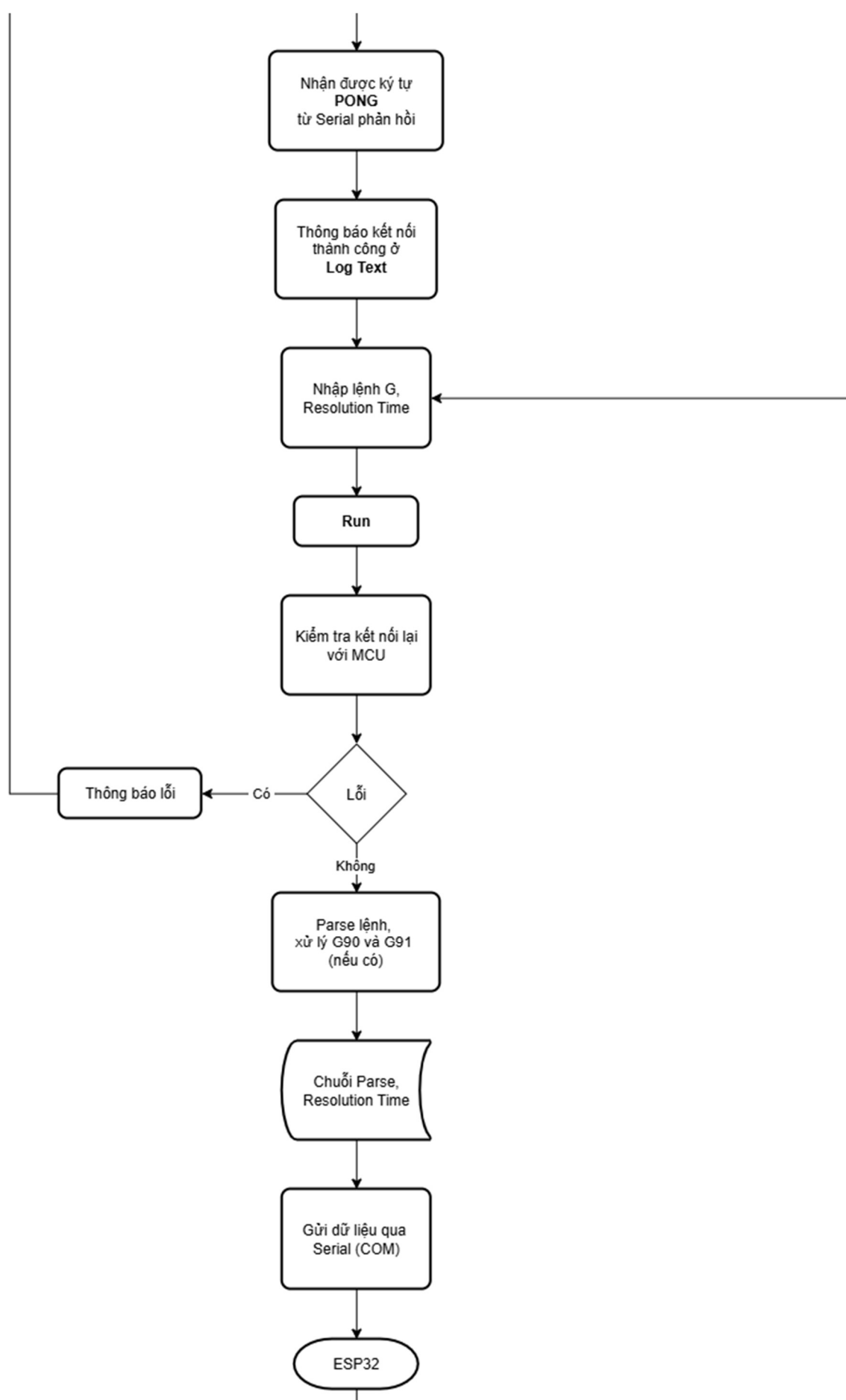
Số thứ tự	Tên thiết bị	Số lượng	Mục đích
1	Máy tính	1	Chạy chương trình GUI, nạp lệnh cho MCU
2	Kit phát triển Wifi BLE SoC ESP32 S3 WeAct ESP32-S3-B N16R8 (Dual-Core)	1	MCU sử dụng Core 0 để phát xung cho động cơ và Core 1 để lượng hóa tọa độ và gửi dữ liệu về chương trình GUI của máy chủ
3	Arduino CNC Shield V3	1	Sử dụng 4 khe cắm driver để lắp các driver động cơ bước A4988
4	A4988 Driver	3	Mạch điều khiển động cơ bước hỗ trợ các vi bước
5	Stepper Motor 42 Nema 17	3	Quan sát kết quả của việc phát xung từ driver
6	Nguồn tổ ong 12V 5A	1	Cấp nguồn cho CNC Shield V3 cho driver A4988 điều khiển động cơ

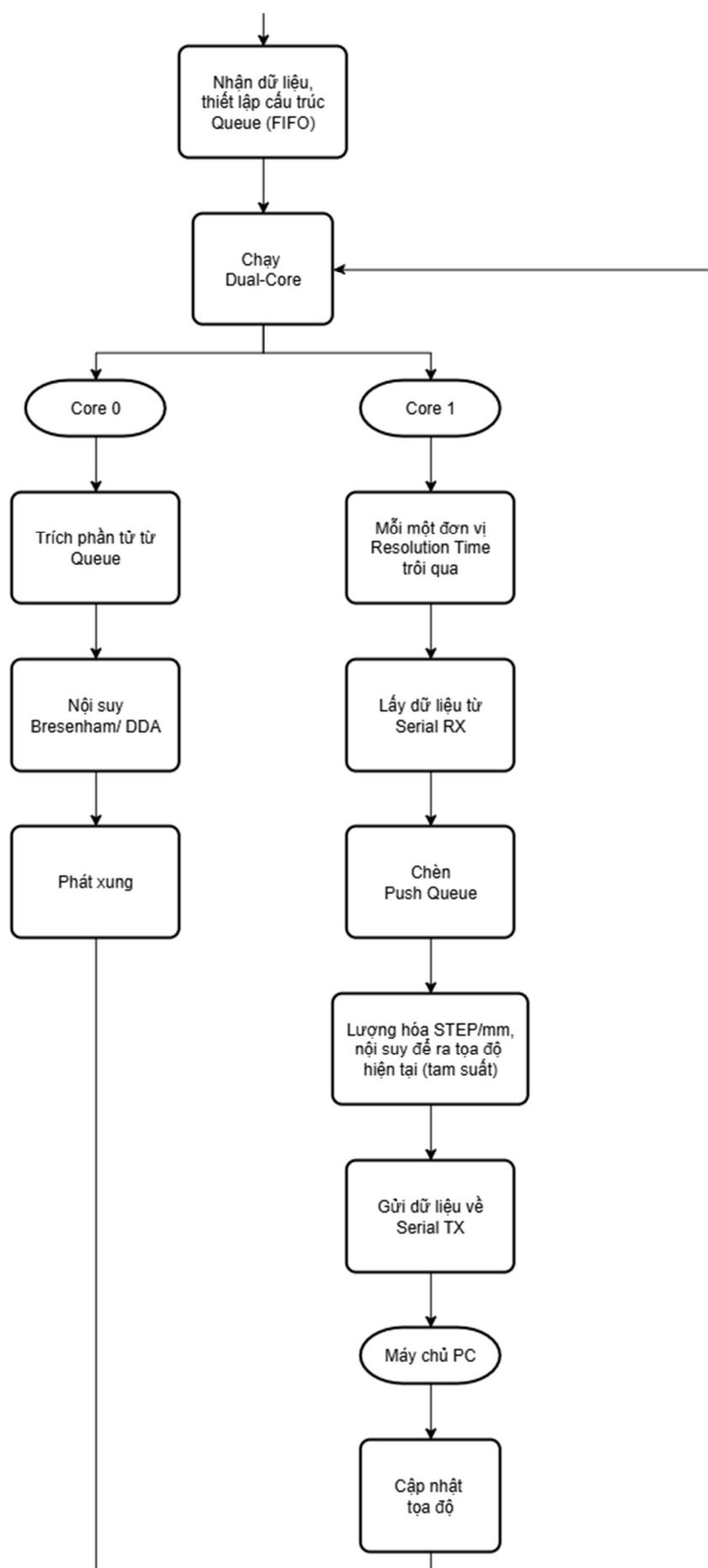
3.6.2. Sơ đồ mạch (Schematic)

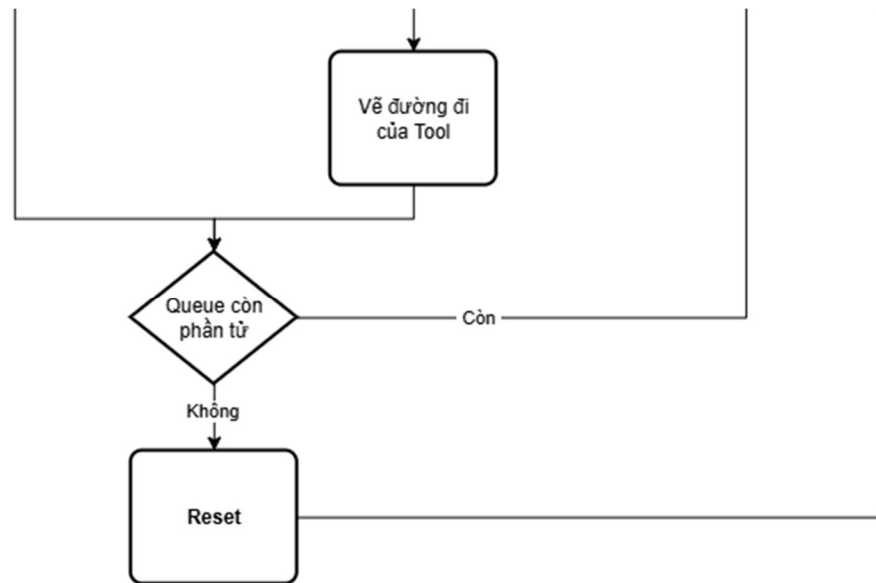


3.7. Thiết kế phần mềm









3.8. Điều khiển động cơ

3.8.1. Timing Lượng hóa STEP trên tọa độ

- **Vấn đề:** vì hiện tại hệ thống không có các thiết bị encoder chuyên dùng nên không phải là closed-loop.
- **Cách giải quyết:** ta sẽ thực hiện lượng hóa một đơn vị STEP/mm để có thể xác định với xung được xuất ra, tọa độ của tool đang ở vị trí nào, giả sử:

- **Motor:** 200 (step/vòng)
- **Vi bước:** 16
- **Vít me:** 8 (mm/vòng)
- Ta có tổng số số step một vòng bao gồm vi bước là:

$$16 \times 200 = 3200 \text{ step/vòng}$$

- Với 3200 step, tương ứng theo vít me, sẽ đi được:

$$3200 \text{ (step)} \leftrightarrow 8 \text{ (mm)}$$

- Như vậy, với 1 step, sẽ đi được:

$$1 \text{ (step)} \leftrightarrow \frac{8}{3200} = \frac{1}{400} \text{ (mm)}$$

, hay

$$400 \text{ (step/mm)}$$

- Từ đây, để có thể lượng hóa được tọa độ, ta sẽ lấy số STEP chia với 400 (step/mm) để có thể lượng hóa ra tọa độ (mm).

3.8.2. Tìm giới hạn Resolution Time

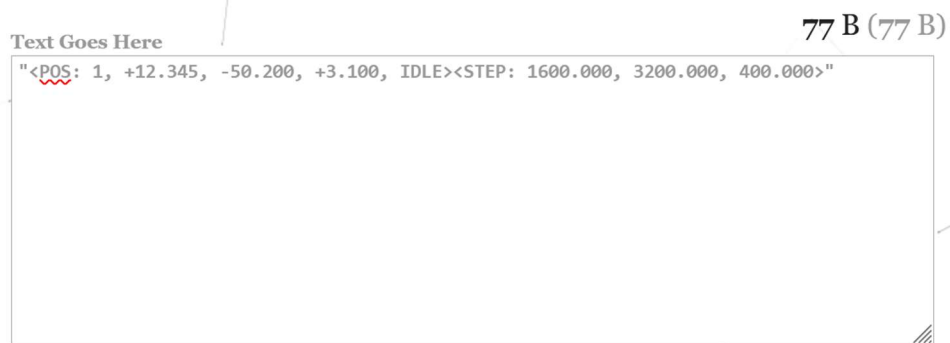
- **Vấn đề:** ta biết với Resolution Time càng nhỏ, GUI có thể vẽ và cập nhật tọa độ của đường đi của tool càng “mượt”. Tuy nhiên, vì ta sử dụng phương thức truyền dữ liệu sang GUI bằng dây USB thông qua kết nối UART với 115200 (baud). Dựa vào thông tin trên, ta có:

- **Baudrate = 115200 (baud):** tốc độ truyền dữ liệu hữu ích, useful byte/s, là:

$$115200 (Bd) = 115200 (bit/s)$$

$$= \frac{115200}{10} (byte/s) = 11520 (byte/s) = 11.52 (KB/s)$$

- **Dung lượng chuỗi phản hồi từ Core 1 của ESP32:**



, như vậy, ta cho rằng một chuỗi thông tin khoảng:

$$80 (Bytes) = 0.08 (KB)$$

- **Giải pháp:** giả sử t là thời gian giới hạn Resolution Time để không vượt quá tốc độ truyền hữu ích (xấp xỉ) $11.52 (KB/s)$, với t được nhập vào từ chương trình GUI, ta có:

$$0.08 (KB) \times \frac{1}{t} (1/s) \leq 11.52 (KB/s)$$

, từ đó ta tính ra:

$$t \geq \frac{1}{144} (s) \approx 0.01 (s) = 10 (ms)$$

- **Kết luận:** vậy Resolution Time, t , cần lớn hơn hoặc bằng $10 (ms)$ để đảm bảo đường truyền Serial hoạt động ổn định.

3.8.3. Thông tin và giới hạn chương trình hiện tại:

- **Chương trình:** chương trình gồm máy chủ, chạy chương trình GUI, được kết nối với MCU, ESP32-S3 qua USB.
- **Giao thức giao tiếp:** Máy chủ và MCU sẽ giao tiếp qua Serial của cáp USB. Dựa trên quá trình tính toán trên, chu kỳ nhỏ nhất của một tín hiệu được truyền từ MCU sang GUI là 0.1 (ms).
- **Giới hạn:**
 - **Lệnh G xử lý:** G0, G1, G90, G91.
 - **Nút chức năng:** các nút Pause, Stop hiện bị vô hiệu hóa.
- **Biến và giá trị:**
 - **Baudrate:** 115200 (baud).
 - **Feed rate mặc định (G0):** 1500 (mm/phút).
 - **Feed rate mặc định (không phải G0, nếu không nhập):** 300 (mm/phút).
- **Cách tính khoảng thời gian phát xung (thời gian delay) dựa trên feedrate:**
 - **Theo lý thuyết (đã chạy và xảy ra lỗi):**

$$Feedrate (mm/min) = \frac{Feedrate}{60} (mm/s)$$

, ta có độ dài quãng đường của 3 trục:

$$Total_Distance = \sqrt{DX^2 + DY^2 + DZ^2}$$

, với giá trị FEEDRATE từ lệnh, ta có tổng thời gian di chuyển của 3 trục là:

$$Total_Time = \frac{Total_Distance (mm)}{\frac{Feedrate}{60} (mm/s)}$$

, vậy thời gian delay lý thuyết mỗi step là:

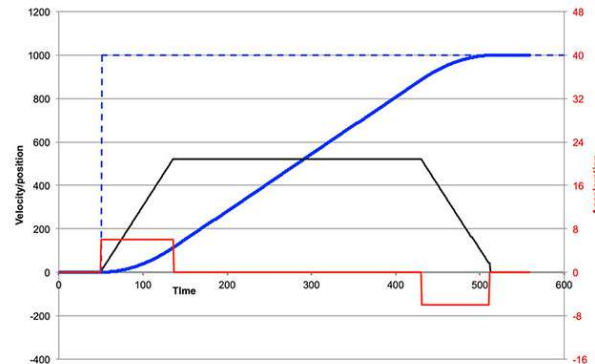
$$Delay_Time = \frac{Total_Time(s)}{MAX_STEP} \times 10^6 (us)$$

○ **Giải pháp:**

Tốc độ càng nhỏ → Thời gian delay càng lớn

Tốc độ càng lớn → Thời gian delay càng nhỏ → dễ trượt bước

- Áp dụng *Trapezoidal Moves Profile*, ta cần xử lý quá trình tăng tốc (*Ramp-up*) và quá trình giảm tốc (*Ramp-down*):



- Dựa vào đồ thị, ta nhận thấy có 3 quá trình: **tăng tốc (ramp-up), chạy đều (cruise), giảm tốc (ramp-down)**.
- Đầu tiên, khi xử lý dòng lệnh, ta cần xác định quãng đường lớn nhất trong 3 trục:

$$MAX_STEP = \max(DX, \max(DY, DZ))$$

, với DX , DY và DZ là quãng đường mà mỗi trục cần di chuyển.

- Tiếp theo, ta cần tính số STEP ở mỗi giai đoạn:

$$RAMP_STEP = MAX_STEP/3$$

○ **Vấn đề:** Tuy nhiên, nếu giả sử:

$$MAX_STEP = 60\,000$$

, vì thế:

$$RAMP_STEP = 20\,000$$

, với số STEP lớn này, việc bắt động cơ tăng tốc trong 20 000 STEP, là lãng phí. Vì thế, ta sẽ đặt một ngưỡng STEP, khi $RAMP_STEP$ vượt qua ngưỡng đó, $RAMP_STEP$ sẽ bằng giá trị ngưỡng đó.

- Với các giá FEEDRATE được nhập vào từ dòng lệnh, chương trình có thể tạm tính giá trị thời gian delay lý thuyết.

$$DELAY_TARGET = \frac{150000}{Feedrate} (us/STEP)$$

- Đồng thời, để tránh hiện tượng bị trượt bước và tuân thủ Trapezoidal Profile, ta gán một giá trị thời gian delay bắt đầu:

$$DELAY_START = 1200 (us)$$

- Chương trình máy CNC, sẽ xem quãng đường của trục có giá trị lớn nhất, từ đó, khi lệnh chạy xong, tất cả các trục phải đến vị trí theo dòng lệnh đang được xử lý cùng lúc.
- Từ đó ta chia thành hai giai đoạn sau:

Trường hợp 1: STEP < RAMP_STEP (Giai đoạn tăng tốc)

$$DELAY = START_{DELAY} - ((START_DELAY - TARGET_DELAY) \times \frac{STEP}{RAMP_STEP})$$

, với STEP là số bước đã thực hiện.

Trường hợp 2: STEP >= MAX_STEP - RAMP_STEP (Giai đoạn giảm tốc)

$$DELAY = START_{DELAY} - ((START_DELAY - TARGET_DELAY) \times \frac{REM_STEP}{RAMP_STEP})$$

, với REM_STEP là số bước còn lại:

$$REM_STEP = MAX_STEP - STEP$$